

Reviewer Pack — Minimal Affine Root Count Correlation with Communication Com...

Entry ecac2a371aa9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 22:43:01 UTC

Minimal Affine Root Count Correlation with Communication Complexity

Entry ID: ecac2a371aa9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 22:43:01 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): Communication Complexity

Statement:

For every instance of a communication complexity problem φ , the minimal number of affine roots ($\text{aff_roots}(\varphi)$) required to represent the solution is linearly correlated with its communication complexity measure $C(\varphi)$, such that $\text{aff_roots}(\varphi) = \Theta(C(\varphi))$.

Rationale (proposer's reasoning):

Affine geometry provides a rich framework for studying geometric configurations and their algebraic properties. The minimal affine root count could capture the intrinsic geometric structure of communication problems, potentially revealing novel insights into the complexity of information transmission.

Taxonomy category: AffineGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7add7315fcf3ff90

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the Pearson correlation coefficient between $\text{aff_roots}(\varphi)$ and $C(\varphi)$ is significant (p-value < 0.05), and at least 80% of seeds show a correlation coefficient ≥ 0.7 , then the conjecture is supported. If any seed does not meet these criteria, or if any seed produces a correlation coefficient ≤ 0.3 , the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal affine root count" AND "communication complexity" - "affine geometry" AND communication complexity AND "root count" - "affine roots" IN("communication complexity", "communication complexity measure")

Top relevant hits considered: - [http://arxiv.org/abs/2210.01601v2] Quantum communication complexity of linear regression - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/2509.22004v3] A Hierarchy for Constant Communication Complexity - [s2:1b50db1680bf5736d34c42fce4bbc970273e8525] Affine-based time-scale ultra wideband wireless channel simulator for time-varying communication environment - [s2:2510.12912] Enabling Full-Duplex ISAC Leveraging Waveform-Domain Separability

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        # Find pivot
        max_row = i
        for j in range(i+1, rows):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below
        for j in range(i+1, rows):
            factor = Fraction(matrix[j][i], matrix[i][i])
            for k in range(cols):
                matrix[j][k] -= factor * matrix[i][k]

    return matrix

def rank(matrix):
    rows, cols = len(matrix), len(matrix[0])
    gaussian_elimination(matrix)
    rank = 0
    for i in range(rows):
```

```

        if any(matrix[i][j] != 0 for j in range(cols)):
            rank += 1
    return rank

def generate_communication_problem(n):
    # Generate a random linear system  $Ax = b$ 
    A = [[random.randint(-1, 1) for _ in range(n)] for _ in range(n)]
    b = [random.randint(-1, 1) for _ in range(n)]
    return A, b

def aff_roots(A, b):
    # Find the minimal number of affine roots
    augmented_matrix = [row + [b[i]] for i, row in enumerate(A)]
    rank_A = rank(augmented_matrix)
    return n - rank_A

def communication_complexity(n):
    # Simplified measure of communication complexity (number of variables)
    return n

def run_trial(seed: int) -> dict:
    random.seed(seed)
    instances_tested = 0
    aff_roots_sum = 0
    comm_complexity_sum = 0
    aff_roots_squared_sum = 0
    comm_complexity_squared_sum = 0

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
            A, b = generate_communication_problem(n)
            aff_roots_val = aff_roots(A, b)
            comm_complexity_val = communication_complexity(n)

            instances_tested += 1
            aff_roots_sum += aff_roots_val
            comm_complexity_sum += comm_complexity_val
            aff_roots_squared_sum += aff_roots_val ** 2
            comm_complexity_squared_sum += comm_complexity_val ** 2

    if instances_tested == 0:
        return {
            "metric_name": "aff_roots vs comm_complexity",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 40,
            "conjecture_holds": False,
            "counterexample": "no_instances_generated"
        }

    aff_roots_mean = Fraction(aff_roots_sum, instances_tested)
    comm_complexity_mean = Fraction(comm_complexity_sum, instances_tested)
    aff_roots_variance = (aff_roots_squared_sum - instances_tested * aff_roots_mean ** 2) / instances_tested
    comm_complexity_variance = (comm_complexity_squared_sum - instances_tested * comm_complexity_mean ** 2) / instances_tested

    if aff_roots_variance == 0 or comm_complexity_variance == 0:

```

```

    return {
        "metric_name": "aff_roots vs comm_complexity",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": 40,
        "conjecture_holds": False,
        "counterexample": "variance_zero"
    }

correlation_coefficient = (instances_tested * aff_roots_sum * comm_complexity_sum - aff_roots_
    math.sqrt(aff_roots_variance * comm_complexity_variance)
)

return {
    "metric_name": "aff_roots vs comm_complexity",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": 40,
    "conjecture_holds": abs(correlation_coefficient) >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif any(not r["conjecture_holds"] for r in results) and any(r["metric_value"] <= 0.3 for r in
        print("RESULT: FALSIFIED counterexample=\"correlation_too_low\" first_failing_seed=1")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_20954889.py", line 125, in <module>
    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_20954889.py", line 72, in run_trial

```

```
aff_roots_val = aff_roots(A, b)
                ^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_20954889.py", line 55, in aff_roots
    return n - rank_A
           ^
```

NameError: name 'n' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Pearson correlation coefficient and its significance. | next: Review the code for errors that may cause a crash and ensure it runs to completion.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18918
2	preregistration	ollama_remote	glm4:latest	0	0	9673
3	novelty	ollama_remote	glm4:latest	0	0	8396
4	novelty	ollama_remote	glm4:latest	0	0	9896
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20873
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9813
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11836
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20434
9	judge	ollama_remote	glm4:latest	0	0	8913

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 118752 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/ecac2a371aa9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ecac2a371aa9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ecac2a371aa9.tar.gz` (if generated)